

Atividade Prática 3 – Display de 7 Segmentos

Pedro Lucas dos Reis Silva - RA: 2272040

Lucas dos Reis Viana - RA: 2321645

12/04/2026

1 Introdução

Esta atividade tem como objetivo a implementação de um decodificador BCD (*Binary-Coded Decimal*) para displays de 7 segmentos na placa FPGA DE10-Lite. Dois números de 0 a 9 são inseridos através das slide switches e apresentados simultaneamente em dois displays de 7 segmentos (SSD – *Seven-Segment Display*).

Um display de 7 segmentos é composto por sete LEDs independentes, nomeados de **a** a **g**, dispostos em formato de “8”, além de um ponto decimal (DP). Na placa DE10-Lite, os displays operam com lógica ativa em nível baixo (ânodo comum): um segmento acende quando o sinal correspondente está em '0' e apaga quando está em '1'. A Figura 1 ilustra a disposição dos segmentos.

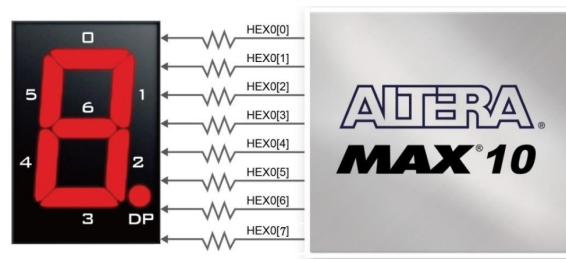


Figura 1: Disposição dos segmentos (a–g) em um display de 7 segmentos.

Para representar cada dígito decimal (0–9), são necessários 4 bits de entrada ($2^4 = 16 > 9$), configurando uma codificação BCD. O decodificador converte cada combinação de 4 bits no padrão de segmentos correspondente.

Nesta atividade, foram introduzidos os seguintes conceitos em VHDL:

- **Tipo `std_logic_vector`:** tipo de dado que representa um vetor (barramento) de bits, permitindo agrupar múltiplos sinais em uma única porta. Diferente do tipo `bit` utilizado nas atividades anteriores, o `std_logic_vector` faz parte da biblioteca `ieee.std_logic_1164` e suporta múltiplos estados lógicos (além de '0' e '1').
- **Atribuição seletiva (`with...select`):** construção concorrente em VHDL que permite mapear valores de entrada a saídas correspondentes, funcionando de forma análoga a um `switch-case` em linguagens de programação. A cláusula `others` trata todos os casos não especificados explicitamente.
- **Nomeação genérica de portas:** as saídas dos displays foram nomeadas como `SSD0` e `SSD1`, permitindo reutilizar o mapeamento de pinos (*Pin Planner*) em atividades futuras.

2 Desenvolvimento

2.1 Descrição do Problema

O circuito recebe 8 bits de entrada através das slide switches da placa DE10-Lite: os 4 bits menos significativos (`sw(3 downto 0)`) representam o primeiro número e controlam o display `SSD0`, enquanto os 4 bits mais significativos (`sw(7 downto 4)`) representam o segundo número e controlam o display `SSD1`. Cada display possui 8 bits de saída: 7 segmentos (g, f, e, d, c, b, a) e o ponto decimal (DP), que permanece sempre desligado ('1').

Para valores de entrada acima de 9 (combinações de "1010" a "1111"), optou-se por exibir a letra "E" (erro) no display, sinalizando que a entrada não corresponde a um dígito decimal válido.

2.2 Mapeamento BCD para 7 Segmentos

A Tabela 1 apresenta o mapeamento completo entre os valores BCD de entrada e os padrões de segmentos correspondentes. O bit mais significativo (bit 7) corresponde ao ponto decimal (DP), sempre '1' (desligado), seguido pelos segmentos g, f, e, d, c, b, a (bits 6 a 0). Lembrando que '0' acende o segmento e '1' apaga.

Tabela 1: Mapeamento BCD → 7 segmentos (DP g f e d c b a).

Entrada (BCD)	Saída (DP g f e d c b a)	Segmentos acesos
0000 (0)	11000000	a, b, c, d, e, f
0001 (1)	11111001	b, c
0010 (2)	10100100	a, b, d, e, g
0011 (3)	10110000	a, b, c, d, g
0100 (4)	10011001	b, c, f, g
0101 (5)	10010010	a, c, d, f, g
0110 (6)	10000010	a, c, d, e, f, g
0111 (7)	11111000	a, b, c
1000 (8)	10000000	a, b, c, d, e, f, g
1001 (9)	10010000	a, b, c, d, f, g
others (> 9)	10000110	a, e, f, g (letra E)

2.3 Implementação VHDL

O código VHDL utiliza a construção `with...select` para implementar o decodificador BCD → 7 segmentos. A entidade `atividade3` possui três portas:

- `sw` (in `std_logic_vector(7 downto 0)`): vetor de 8 bits representando as 8 slide switches de entrada.
- `SSD0` (out `std_logic_vector(7 downto 0)`): saída de 8 bits para o primeiro display.
- `SSD1` (out `std_logic_vector(7 downto 0)`): saída de 8 bits para o segundo display.

Na arquitetura, o vetor `sw` é fatiado (*slicing*) em dois grupos de 4 bits: `sw(3 downto 0)` seleciona o padrão para `SSD0` e `sw(7 downto 4)` seleciona o padrão para `SSD1`. Cada bloco

`with...select` funciona como um multiplexador, mapeando cada valor BCD ao padrão de segmentos correspondente. A cláusula `when others` garante que qualquer valor fora do intervalo 0–9 exiba o caractere de erro “E”.

O código VHDL completo encontra-se no Apêndice A ao final deste relatório.

2.4 Pin Planner

O mapeamento entre os sinais VHDL e os pinos físicos da placa DE10-Lite foi realizado através do *Pin Planner* do Quartus Prime. A Tabela 2 apresenta os pinos utilizados, e a Figura 2 mostra a listagem completa no Quartus.

Tabela 2: Pin Planner – mapeamento dos sinais para a DE10-Lite.

Sinal VHDL	Componente	Pino FPGA	I/O Standard
sw[0]	SW0 (Slide Switch 0)	PIN_C10	3.3-V LVTTL
sw[1]	SW1 (Slide Switch 1)	PIN_C11	3.3-V LVTTL
sw[2]	SW2 (Slide Switch 2)	PIN_D12	3.3-V LVTTL
sw[3]	SW3 (Slide Switch 3)	PIN_C12	3.3-V LVTTL
sw[4]	SW4 (Slide Switch 4)	PIN_A12	3.3-V LVTTL
sw[5]	SW5 (Slide Switch 5)	PIN_B12	3.3-V LVTTL
sw[6]	SW6 (Slide Switch 6)	PIN_A13	3.3-V LVTTL
sw[7]	SW7 (Slide Switch 7)	PIN_A14	3.3-V LVTTL
SSD0[0]	HEX0 segmento a	PIN_C14	3.3-V LVTTL
SSD0[1]	HEX0 segmento b	PIN_E15	3.3-V LVTTL
SSD0[2]	HEX0 segmento c	PIN_C15	3.3-V LVTTL
SSD0[3]	HEX0 segmento d	PIN_C16	3.3-V LVTTL
SSD0[4]	HEX0 segmento e	PIN_E16	3.3-V LVTTL
SSD0[5]	HEX0 segmento f	PIN_D17	3.3-V LVTTL
SSD0[6]	HEX0 segmento g	PIN_C17	3.3-V LVTTL
SSD0[7]	HEX0 ponto decimal	PIN_D15	3.3-V LVTTL
SSD1[0]	HEX1 segmento a	PIN_C18	3.3-V LVTTL
SSD1[1]	HEX1 segmento b	PIN_D18	3.3-V LVTTL
SSD1[2]	HEX1 segmento c	PIN_E18	3.3-V LVTTL
SSD1[3]	HEX1 segmento d	PIN_B16	3.3-V LVTTL
SSD1[4]	HEX1 segmento e	PIN_A17	3.3-V LVTTL
SSD1[5]	HEX1 segmento f	PIN_A18	3.3-V LVTTL
SSD1[6]	HEX1 segmento g	PIN_B17	3.3-V LVTTL
SSD1[7]	HEX1 ponto decimal	PIN_A16	3.3-V LVTTL

Node Name	Direction	Location	I/O Bank	VREF Group	Fitter Location	I/O Standard
out SSD0[7]	Output				PIN_V12	2.5 V (default)
out SSD0[6]	Output	PIN_C17	7	B7_N0	PIN_C17	2.5 V
out SSD0[5]	Output	PIN_D17	7	B7_N0	PIN_D17	2.5 V
out SSD0[4]	Output	PIN_E16	7	B7_N0	PIN_E16	2.5 V
out SSD0[3]	Output	PIN_C16	7	B7_N0	PIN_C16	2.5 V
out SSD0[2]	Output	PIN_C15	7	B7_N0	PIN_C15	2.5 V
out SSD0[1]	Output	PIN_E15	7	B7_N0	PIN_E15	2.5 V
out SSD0[0]	Output	PIN_C14	7	B7_N0	PIN_C14	2.5 V
out SSD1[7]	Output				PIN_F2	2.5 V (default)
out SSD1[6]	Output	PIN_B17	7	B7_N0	PIN_B17	2.5 V
out SSD1[5]	Output	PIN_A18	7	B7_N0	PIN_A18	2.5 V
out SSD1[4]	Output	PIN_A17	7	B7_N0	PIN_A17	2.5 V
out SSD1[3]	Output	PIN_B16	7	B7_N0	PIN_B16	2.5 V
out SSD1[2]	Output	PIN_E18	6	B6_N0	PIN_E18	2.5 V
out SSD1[1]	Output	PIN_D18	6	B6_N0	PIN_D18	2.5 V
out SSD1[0]	Output	PIN_C18	7	B7_N0	PIN_C18	2.5 V
in sw[7]	Input	PIN_A14	7	B7_N0	PIN_A14	2.5 V
in sw[6]	Input	PIN_A13	7	B7_N0	PIN_A13	2.5 V
in sw[5]	Input	PIN_B12	7	B7_N0	PIN_B12	2.5 V
in sw[4]	Input	PIN_A12	7	B7_N0	PIN_A12	2.5 V
in sw[3]	Input	PIN_C12	7	B7_N0	PIN_C12	2.5 V
in sw[2]	Input	PIN_D12	7	B7_N0	PIN_D12	2.5 V
in sw[1]	Input	PIN_C11	7	B7_N0	PIN_C11	2.5 V
in sw[0]	Input	PIN_C10	7	B7_N0	PIN_C10	2.5 V

Figura 2: Listagem do Pin Planner no Quartus Prime.

2.5 Foto da Placa

A Figura 3 apresenta a placa DE10-Lite com os displays de 7 segmentos exibindo os números selecionados pelas slide switches.

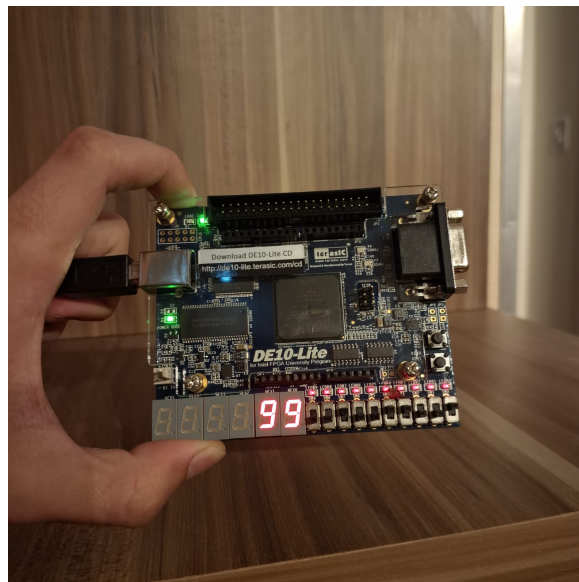


Figura 3: Placa DE10-Lite com os displays de 7 segmentos em funcionamento.

2.6 Diagrama RTL

O diagrama RTL foi gerado pelo Quartus Prime através da ferramenta *RTL Viewer* (Tools → Netlist Viewers → RTL Viewer). A Figura 4 apresenta a estrutura do circuito sintetizado, onde se observam os dois multiplexadores correspondentes aos blocos `with...select`, cada um mapeando 4 bits de entrada nos 8 bits de saída do respectivo display.

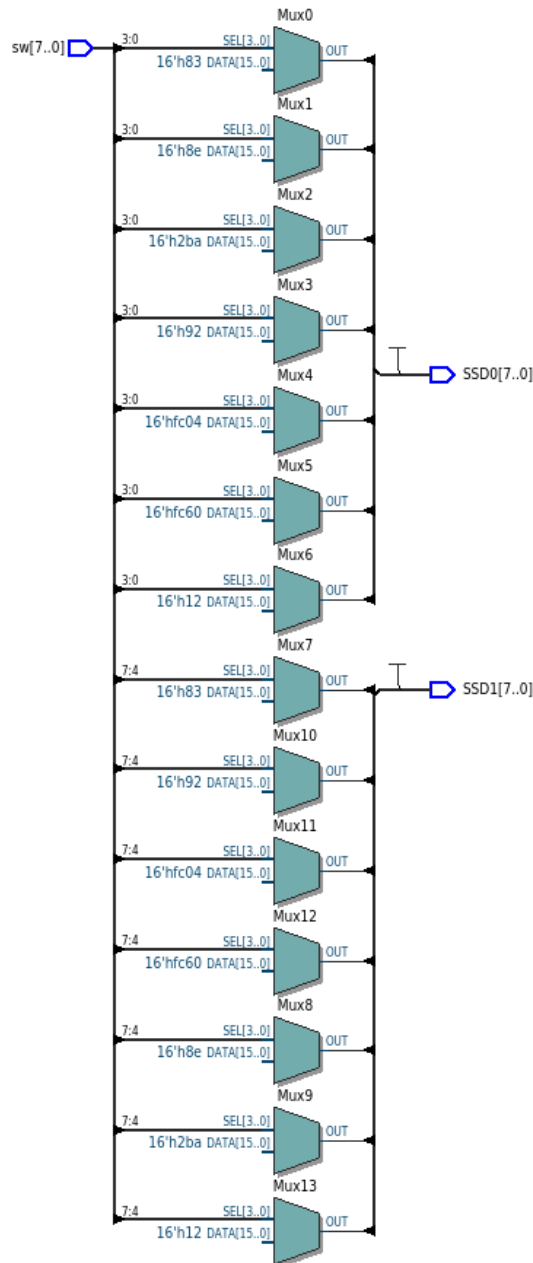


Figura 4: Diagrama RTL do decodificador BCD para 7 segmentos, gerado pelo Quartus Prime.

3 Considerações Finais

O decodificador BCD para displays de 7 segmentos foi implementado com sucesso na placa DE10-Lite. O circuito converte corretamente os valores de 0 a 9, inseridos via slide switches, nos padrões de segmentos correspondentes, exibindo os dígitos nos displays HEX0 e HEX1. Para valores acima de 9, o display exibe a letra “E”, indicando entrada inválida.

A utilização do tipo `std_logic_vector` permitiu agrupar os sinais em barramentos, simplificando a descrição das portas e a atribuição dos padrões de segmentos. A construção `with...select` mostrou-se adequada para implementar a lógica de decodificação de forma concisa e legível, mapeando diretamente cada valor BCD ao padrão de saída desejado. A nomeação genérica das portas dos displays (SSD0, SSD1) possibilita a reutilização do mapeamento de pinos em atividades futuras.

A Código VHDL – Decodificador BCD para 7 Segmentos

```

1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity atividade3 is
5     port (
6         -- 8 chaves: sw(3 downto 0) para SSD0 e sw(7 downto 4) para
           SSD1
7         sw    : in  std_logic_vector(7 downto 0);
8         -- Saidas genericas para os displays de 7 segmentos
9         -- Formato: (DP g f e d c b a) - 8 bits, DP sempre '1' (
           desligado)
10        SSD0 : out std_logic_vector(7 downto 0);
11        SSD1 : out std_logic_vector(7 downto 0)
12    );
13 end entity atividade3;
14
15 architecture ssds of atividade3 is
16 begin
17
18     -- Decodificador BCD para 7 segmentos - SSD0
19     -- Chaves sw(3 downto 0) controlam o primeiro display
20     -- Segmentos acendem com nivel '0' (anodo comum)
21     -- Bit 7 = DP (ponto decimal, sempre desligado = '1')
22     -- Bits 6..0 = g f e d c b a
23     with sw(3 downto 0) select
24         SSD0 <= "11000000" when "0000", -- 0: acende a,b,c,d,e,f
25                "11111001" when "0001", -- 1: acende b,c
26                "10100100" when "0010", -- 2: acende a,b,d,e,g
27                "10110000" when "0011", -- 3: acende a,b,c,d,g
28                "10011001" when "0100", -- 4: acende b,c,f,g
29                "10010010" when "0101", -- 5: acende a,c,d,f,g
30                "10000010" when "0110", -- 6: acende a,c,d,e,f,g
31                "11111000" when "0111", -- 7: acende a,b,c
32                "10000000" when "1000", -- 8: acende todos
33                "10010000" when "1001", -- 9: acende a,b,c,d,f,g
34                "10000110" when others; -- E (erro) para valores > 9
35
36     -- Decodificador BCD para 7 segmentos - SSD1
37     -- Chaves sw(7 downto 4) controlam o segundo display
38     with sw(7 downto 4) select
39         SSD1 <= "11000000" when "0000", -- 0
40                "11111001" when "0001", -- 1
41                "10100100" when "0010", -- 2
42                "10110000" when "0011", -- 3
43                "10011001" when "0100", -- 4
44                "10010010" when "0101", -- 5
45                "10000010" when "0110", -- 6
46                "11111000" when "0111", -- 7
47                "10000000" when "1000", -- 8

```

```

48         "10010000" when "1001", -- 9
49         "10000110" when others; -- E (erro) para valores > 9
50
51 end architecture ssds;
```

Listing 1: Arquivo atividade3.vhd – decodificador BCD para dois displays de 7 segmentos.